L Autoencoder hyperparameters

The hyperparameter setup for all the autoencoder architectures is detailed in Table 5. All autoencoder architectures are trained using pytorch-lightning and the input data is normalized using a MinMaxScaler.

Table 5: Autoencoder Hyperparameters. All four architectures used the same hyperparameters. To ensure similarity of testing conditions we replicate the hyperparameter setup from Trofimov et al. (2023)

Dataset Name	Batch Size	LR	Hidden Dim	Layers	Epochs	Metric Start Epoch
MNIST	256	10^-4	512	3	250	60
F-MNIST	256	10^-4	512	3	250	60
CIFAR-10	256	10^-4	512	3	250	60
COIL-20	256	10^-4	512	3	250	60

M GNN hyperparameters

The hyperparameter setup for all the GNN architectures is detailed in Table 6

Table 6: GNN Architecture Information

Architecture	Layers	Hidden Dim	LR	Epochs	Additional Info
GCN	4	128	0.001	200	-
GraphSAGE	4	128	0.001	200	Mean Aggregation
GAT	4	128	0.001	200	8 heads with Hid Dim 8 each
CGCN	4	128	0.001	200	8 subgraphs

N GNN metrics

The test accuracy for node classification and the ROC AUC value for link prediction is detailed in Table 7

Table 7: GNN Metric Data on Amazon Computer Dataset. Test Accuracy is for Node Classification and ROC AUC Score is for Link Prediction

Architecture	Accuracy (NC)	ROC AUC (LP)
GCN	0.8257	0.8638
GraphSAGE	0.8800	0.8068
GAT	0.8273	0.7997
CGCN	0.8200	0.8716

O Datasets

We report the statistics of the datasets used for the empirical analysis of GNNs in Table 8. We report the exact statistics of the autoencoder datasets in Table 9.

O.1 Graph Datasets

The Amazon computers dataset is a subset of the Amazon co-purchase graph (Shchur et al., 2019). The nodes represent products on amazon and the edges indicate that two products are frequently bought together. The node features are the product reviews encoded in bag-of-words format. The class labels represent the product category. We also use the Flickr, Cora, Citeseer and Pubmed dataset in additional experiments.

Table 8: Dataset Statistics for Graph

Dataset	**Amazon Computer**	**Flickr**	**Cora**	**Citeseer**	**Pubmed**
Number of Nodes	13752	89250	2708	3327	19717
Number of Edges	491722	899756	10556	9104	88648
Average Degree	35.76	10.08	3.90	2.74	4.50
Node Features	767	500	1433	3703	500
Labels	10	7	7	6	3

O.2 AutoEncoder Datasets

Four diverse real-world datasets were utilized for our experiments: MNIST (LeCun et al., 2010), Fashion-MNIST (F-MNIST) (Xiao et al., 2017), COIL-20 (Nene et al., 1996), and CIFAR-10 (Krizhevsky & Hinton, 2009), to comprehensively evaluate the performance of our autoencoder model. MNIST, comprising 28x28 grayscale images of handwritten digits, serves as a foundational benchmark for image classification and feature extraction tasks. Fashion-MNIST extends this by offering a similar format but with 10 classes of clothing items, making it an ideal choice for fashion-related image analysis. COIL-20 presents a unique challenge, with 20 object categories, where each category consists of 72 128x128 color images captured from varying viewpoints, offering a more complex 3D object recognition scenario.

Table 9: Dataset Statistics for AE

Dataset	**Classes**	**Train Size**	**Test Size**	**Image Size**	**Data Type**
MNIST	10	60,000	10,000	28x28 (784)	Grayscale
Fashion-MNIST (F-MNIST)	10	60,000	10,000	28x28 (784)	Grayscale
COIL-20	20	1,440	-	128x128 (16384)	Color
CIFAR-10	10	60,000	10,000	32x32*3 (3072)	Color

P Ablation Study

NSA consists of two parts, GlobalNSA and LocalNSA. These two components work well together and are capable of working individually too. LocalNSA makes locally optimal decisions but it could run into a problem when the global structure needs to be preserved. Figure 8 shows a scenario where LocalNSA would not obtain a high k-NN consistency between two representation spaces. This could potentially explain the low kNN consistency when minimizing with just LocalNSA in Table 10. LocalNSA's inability to preserve global structure becomes especially prevalent in our LinkPrediction tests in Table 11. We intend on further improving LocalNSA and coming up with better preprocessing schemes to optimize its performence. In practice, GNSA works well and performs competitively even when used without LNSA. LNSA provides a noticeable improvement but requires dataset specific fine tuning to find the optimal hyperparameter set up.

Table 10: Autoencoder results. NSA-AE outperforms or almost matches all other approaches on all the evaluation metrics. RTD-AE, which explicitly minimizes on RTD has a slightly lower RTD value while PCA has marginally higher Triplet Ranking Accuracy on Cluster Centers.

		Quality measure						
Dataset	Method	L. C.	T. A.	RTD	T. A. C.C	NSA	LID-NSA	kNN-C
MNIST	GNSA-AE	0.942	0.885 ± 0.006	5.44 ± 0.12	0.944 ± 0.230	$\mathbf{0.0198 \pm 0.0001}$	0.0342 ± 0.00	0.616
	LNSA-AE	0.819	0.794 ± 0.007	7.39 ± 0.21	0.832 ± 0.374	0.0548 ± 0.00	0.0373 ± 0.00	0.607
	NSA-AE	**0.947**	$\mathbf{0.886 \pm 0.005}$	$\mathbf{4.29 \pm 0.11}$	$\mathbf{0.946 \pm 0.226}$	0.0222 ± 0.00	$\mathbf{0.0123 \pm 0.00}$	**0.634**
F-MNIST	GNSA-AE	**0.987**	$\mathbf{0.952 \pm 0.002}$	4.11 ± 0.21	0.992 ± 0.089	$\mathbf{0.0091 \pm 0.0001}$	0.0350 ± 0.00	0.592
	LNSA-AE	0.887	0.859 ± 0.006	5.41 ± 0.18	0.952 ± 0.214	0.0488 ± 0.00	0.0470 ± 0.00	0.608
	NSA-AE	0.9854	0.939 ± 0.003	$\mathbf{2.78 \pm 0.09}$	$\mathbf{0.992 \pm 0.089}$	0.0114 ± 0.00	$\mathbf{0.0133 \pm 0.00}$	**0.633**
CIFAR-10	GNSA-AE	**0.985**	0.936 ± 0.004	3.07 ± 0.11	0.984 ± 0.125	$\mathbf{0.0077 \pm 0.0001}$	0.0122 ± 0.00	0.453
	LNSA-AE	0.8616	0.849 ± 0.006	4.02 ± 0.28	0.936 ± 0.245	0.0624 ± 0.00	0.0220 ± 0.00	0.359
	NSA-AE	0.985	$\mathbf{0.941 \pm 0.003}$	$\mathbf{3.04 \pm 0.16}$	$\mathbf{0.984 \pm 0.125}$	$\mathbf{0.0086 \pm 0.00}$	$\mathbf{0.0103 \pm 0.00}$	**0.437**
COIL-20	GNSA-AE	0.955	0.919 ± 0.004	7.46 ± 0.23	0.939 ± 0.240	0.0157 ± 0.0	0.3427 ± 0.00	0.721
	LNSA-AE	0.848	0.838 ± 0.007	9.74 ± 0.26	0.887 ± 0.316	0.0672 ± 0.00	0.5810 ± 0.00	0.703
	NSA-AE	**0.976**	$\mathbf{0.919 \pm 0.005}$	$\mathbf{3.55 \pm 0.18}$	$\mathbf{0.962 \pm 0.191}$	$\mathbf{0.0139 \pm 0.00}$	$\mathbf{0.0085 \pm 0.00}$	**0.809**

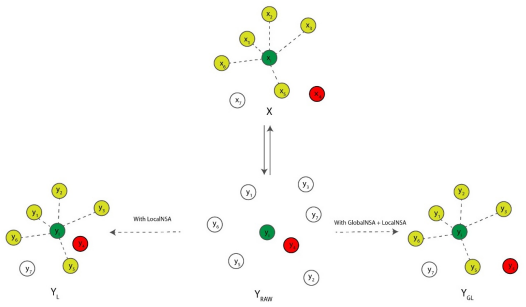

Figure 8: Illustration showing why LocalNSA might not maintain neighborhood consistency when used without GlobalNSA. The objective is to minimize the structural discrepancy between the reference space X (top) and generated space Y (mid-bottom). LocalNSA aims to minimize the difference in distances for the k-nearest neighbors of the ith point in X space (in yellow). If a random initialization causes an unknown point (in red) to move close to the ith point in Y space, LocalNSA will not move it out, instead focusing on bringing the 5 NNs of X close to i in Y causing a drop in neighborhood consistency Y_L (left-bottom). Using GlobalNSA with LocalNSA alleviates this issue as GlobalNSA pushes point 4 away in the Y_{GL} (right-bottom) .

Table 11: Downstream task analysis with Link Prediction. The output embeddings from a GCN trained on link prediction are passed through all 3 autoencoder architectures. Latent embeddings are obtained at various dimensions and ROC-AUC scores are calculated on the latent embeddings. Using both LNSA and GNSA gives the best performance.

		Dataset			
Method	Latent Dim	Amazon Comp	Cora	Citeseer	Pubmed
G-NSA-AE	128	95.74	96.04	97.18	97.31
	64	**95.90**	95.54	96.85	96.40
	32	**96.01**	95.37	94.25	97.53
L-NSA-AE	128	76.82	50.01	50.13	93.03
	64	74.24	50.69	50.25	93.64
	32	80.67	55.44	51.43	95.69
NSA-AE	128	**95.75**	**97.83**	**97.91**	**97.37**
	64	94.60	**97.89**	**98.38**	**96.97**
	32	95.76	**97.68**	**96.11**	**97.66**

P.1 Visualizing The Effect with Spheres Dataset

We use the Spheres dataset (Moor et al., 2020), reducing the original 100 dimension data to 2 dimensions to visually observe the effect that different scaling factors for GNSA and LNSA have. The model is trained on a batch size of 200 and with no rescaling applied to the Spheres dataset. Additionally, NSA is the only loss applied here while in the autoencoder experiments, we also use